

This lecture provides a comprehensive introduction to functions in Python, covering their creation, usage, and various advanced concepts. The main topics discussed include:

- Introduction to Functions:
 - Functions are reusable blocks of code that perform specific tasks.
 - Python has many built-in functions like `len()`, `print()`, `range()`, and `list()`.
 - The lecture focuses on creating custom functions to encapsulate business logic.
- Defining Functions:
 - Functions are defined using the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`.
 - The function body contains the code to be executed when the function is called.
 - The `pass` keyword can be used as a placeholder when a function body is not yet implemented.
- Calling Functions:
 - Functions are called by using their name followed by parentheses `()`.
 - Calling a function executes the code within its body.
- Return Statements:
 - The `return` statement is used to specify the value that a function should return.
 - Functions should use `return` to provide output, rather than `print`, to allow the output to be used in further computations.
 - The `print` function returns `None` type, which cannot be used for primitive operations.
 - A function can return multiple values, which are then returned as a tuple.
- Function Arguments:
 - Functions can accept arguments (inputs) that are passed when the function is called.
 - Arguments are defined within the parentheses `()` in the function definition.
 - When calling a function, you must provide the correct number of arguments as defined in the function signature.
 - Arguments can be passed positionally or by keyword (e.g., `a=5, b=6`).
- Default Argument Values:
 - Function arguments can have default values, which are used if the argument is not provided when the function is called.
 - Default values are specified in the function definition using the `=` operator (e.g., `discount=0.1`).
- `*args` and `**kwargs`:
 - `*args` allows a function to accept any number of positional arguments. These arguments are passed as a tuple.
 - `**kwargs` allows a function to accept any number of keyword arguments. These arguments are passed as a dictionary.
 - `args` and `kwargs` are not keywords; other variable names can be used, but `*` and `**` are essential.
- Docstrings:
 - Docstrings are multiline strings used to document functions.

- They are defined using triple quotes `"""` at the beginning of the function body.
- Docstrings provide information about the function's purpose, arguments, and return value.
- Docstrings are displayed when using the `help()` function or hovering over the function name in some IDEs.
- Lambda Functions:
 - Lambda functions are anonymous, small, and single-expression functions.
 - They are defined using the `lambda` keyword, followed by the arguments, a colon `:`, and the expression to be returned.
 - Lambda functions are often used for short, simple operations.
 - Lambda functions can be stored in variables and called like regular functions.
- Map Function:
 - The `map()` function applies a given function to each item in an iterable (e.g., a list) and returns a map object (iterator).
 - `map()` takes two arguments: the function to apply and the iterable to process.
 - Lambda functions are often used with `map()` for concise operations.
- Filter Function:
 - The `filter()` function filters elements from an iterable based on a given function.
 - `filter()` takes two arguments: the function to use for filtering and the iterable to process.
 - The function should return `True` for elements to be included and `False` for elements to be excluded.
- Reduce Function:
 - The `reduce()` function applies a function cumulatively to the items of an iterable, reducing it to a single value.
 - `reduce()` is available in the `functools` module and needs to be imported.
 - `reduce()` takes two arguments: the function to apply and the iterable to process.
 - The function should take two arguments and return a single value.
- Recursive Functions:
 - Recursive functions are functions that call themselves.
 - Recursive functions must have a base case to stop the recursion and prevent infinite loops.
 - The lecture provides examples of recursive functions for calculating factorials and Fibonacci numbers.
 - The importance of defining a base case and a recursive step is emphasized.
- Fibonacci Sequence:
 - The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones (e.g., 1, 1, 2, 3, 5, 8).
 - The lecture demonstrates how to generate Fibonacci numbers using a recursive function.
- Key Takeaways:
 - Functions promote code reusability, modularity, and maintainability.
 - Python offers various ways to define and use functions, including lambda functions and recursive functions.
 - Understanding `*args` and `**kwargs` allows for flexible function design.

- `map()`, `filter()`, and `reduce()` are powerful tools for processing iterables.
- Recursive functions can be used to solve problems that can be broken down into smaller, self-similar subproblems.

-